

Document Number: P2188R1
Date: 2020-07-15
Author: [Anthony Williams](#)
Just Software Solutions Ltd
Audience: EWG

P2188R1: Zap the Zap: Pointers are sometimes just bags of bits

This paper relates to [P1726: Pointer lifetime-end zap and provenance, too](#). It is not a "competing" paper, but provides an alternative look at the same issues.

R0 of this paper received extensive discussion on the EWG reflector. In light of that discussion, I have updated this paper to make it clear which aspects I believe are important, and how it can work with provenance-based analysis.

All the examples are based on production code that I have seen in use throughout my career. They all "work" with existing code, but I understand that slight variations on them may be optimized in ways that no longer work by current compilers.

One of the baseline goals of C++ is backwards compatibility: don't break users' code unnecessarily. I would therefore like to see a way to preserve as much of these as possible, without unnecessarily hindering optimizers, and while making sure that those that are broken break noisily, so users can more easily migrate to an alternative approach.

The standard says that pointers are *scalar types* and *trivially copyable* types. Consequently the "pointer zap" from the final sentence of [basic.stc] p4 ("Any other use of an invalid pointer value has implementation-defined behavior"), and especially note 31 ("Some implementations might define that copying an invalid pointer value causes a system-generated runtime fault.") is clearly incompatible with pointers being *trivially copyable* types from [basic.types] p3, since that indicates that the value of a pointer is entirely derived from its value representation.

Consequently we need to do *something* to make it clear what is permitted of users and optimizers.

I have removed the proposed wording from this paper, as I don't think it addresses all the issues.

Important aspects

1. Pointers that compare equal should be interchangeable

Pointers have *value semantics* rather than *identity semantics*: if I copy a pointer it is equivalent to the original. An important aspect of that is that if two values are equal they can be used interchangeably.

```
void f(int* p, int* q){
    *q=99;
    bool same=false;
    if(p==q){
        *p=42;
        same=true;
        assert(*q==42);
    }
```

```

    assert(same?(*q==42):(*q==99));
}

```

Assuming p and q are not *uninitialized* values, and q is known to point somewhere valid, this code should work. If it doesn't, then equality is broken for pointers.

I understand that if p and q have different provenance then the compiler might want to treat them differently. That's fine, but if they are to be treated differently then they should not compare equal.

2. Pointer equality is consistent

If I compare two pointers in one place and then compare the same two pointers, or copies of them, in another place, then comparisons must yield the same result. Again, this is a fundamental aspect of *value semantics*.

```

bool compare(int* const p, int* const q){
    return p==q;
}

void f(int* const p, int* const q){
    bool const same=(p==q);
    g(p,q);
    assert(same==(p==q));
    assert(same==compare(p,q));
}

```

Assuming p and q are not *uninitialized* values, this code should work, irrespective of what g does. Either the values are the same, or they are not. This must work across translation units and whether or not there is inlining, otherwise equality is broken for pointers.

3. Compilers must be able to assume no aliasing in certain circumstances

If a function has a local variable x that is not passed to other functions by pointer or reference, then the compiler should be able to assume that variable is unchanged by calls to other functions.

```

void f(){
    int x=42;
    g();
    assert(x==42);
}

```

The assert should never fire, and the compiler should be able to assume that to be the case.

To my mind, this is still consistent with my other points: there are no pointers to x, so any pointers used in g cannot be equal to them.

4. Dereferencing invalid pointers is still undefined behaviour

If I have a pointer p to an object which is destroyed by whatever means, such as the object it points to going out of scope, or being deleted, then dereferencing it is undefined behaviour.

Likewise, if the pointer is otherwise invalid, such as being a one-past-the-end pointer, then dereferencing it is undefined behaviour.

However, if the pointer `p` is compared to another pointer `q` then the compiler may have to integrate the provenance of `q` with the provenance of `p` if `q` is known to be valid, and they compare equal.

5. Validity of pointers is contagious after comparison

If I compare it to another pointer `q` which I know to be valid, and they are equal, then `p` must now be valid too, as per my point 1 above.

```
void f(){
    int * const p=new int(42);
    delete p;
    int * const q=new int(99);

    if(p==q){
        assert(*p==99);
    }
}
```

It is perfectly acceptable to me that `p==q` returns false, and I would encourage implementers to try to ensure that it does. However, if it returns true then `p` must now be assumed to point to the same object as `q`.

In sequential code this seems contrived and unlikely, but in concurrent code such as several of the examples from [P1726](#) this can be important.

Trivially-Copyable Examples

Example 1: memcpy on a pointer

[Compiler explorer link](#)

```
#include <assert.h>
#include <string.h>

int main() {
    int *x= new int(42);
    int *y= nullptr;
    memcpy(&y,&x,sizeof(x));
    assert(x == y);
    assert(*y==42);
}
```

Here, we use `memcpy` to copy the bits of a pointer from one pointer to another. The second pointer is now valid and points to the same thing the original did because pointers are *trivially copyable* ([basic.types] p3).

Example 2: memcpy via a buffer

[Compiler explorer link](#)

```
#include <assert.h>
#include <string.h>

int main() {
    int *x= new int(42);
    int *y= nullptr;
    unsigned char buffer[sizeof(x)];
    memcpy(buffer, &x, sizeof(x));
}
```

```

    memcpy(&y, buffer, sizeof(x));
    assert(x == y);
    assert(*y == 42);
}

```

Here, we use `memcpy` to copy the bits of a pointer from one pointer to a buffer, and then from that buffer to another pointer. The second pointer is now valid and points to the same thing the original did because pointers are *trivially copyable* ([basic.types] p2).

Example 3: reinterpret_cast to an integer

[Compiler explorer link](#)

```

#include <assert.h>
#include <string.h>
#include <stdint.h>

int main() {
    int *x= new int(42);
    int *y= nullptr;
    uintptr_t temp= reinterpret_cast<uintptr_t>(x);
    y= reinterpret_cast<int *>(temp);
    assert(x == y);
    assert(*y == 42);
}

```

Here we rely on the provision of [expr.reinterpret.cast] p5 that a pointer may be cast to an integer and back and retain its value.

Example 4: memcpy with modifications

[Compiler explorer link](#)

```

#include <assert.h>
#include <string.h>

int main() {
    int *x= new int(42);
    int *y= nullptr;
    unsigned char buffer[sizeof(x)];
    memcpy(buffer, &x, sizeof(x));
    for(auto &c : buffer) {
        c ^= 0x55;
    }
    for(auto &c : buffer) {
        c ^= 0x55;
    }
    memcpy(&y, buffer, sizeof(x));
    assert(x == y);
    assert(*y == 42);
}

```

Now we take example 1 a step further: we perform a reversible modification on the bits in the buffer after the first `memcpy`, then reverse that modification and `memcpy` it back. Since the bits in the buffer now hold their original values, we can copy them to a pointer, which will have the same value, because pointers are *trivially copyable*.

Example 5: memcpy and write to file

[Compiler explorer link](#)

```
#include <assert.h>
#include <string.h>
#include <stdio.h>

int main() {
    int *x= new int(42);
    int *y= nullptr;
    unsigned char buffer[sizeof(x)];
    memcpy(buffer, &x, sizeof(x));

    auto file= fopen("tempfile", "wb");
    auto written= fwrite(buffer, 1, sizeof(buffer), file);
    assert(written == sizeof(buffer));
    fclose(file);

    memset(buffer, 0, sizeof(buffer));

    file= fopen("tempfile", "rb");
    auto read= fread(buffer, 1, sizeof(buffer), file);
    assert(read == sizeof(buffer));
    fclose(file);
    memcpy(&y, buffer, sizeof(x));
    assert(x == y);
    assert(*y == 42);
}
```

This time we are copying the pointer to a buffer, writing our bytes to a file, clearing the buffer and reading the bytes back from the file, then copying the bytes back to the pointer. If our file is unmodified then the buffer will have the same contents after reading as it did before writing, so copying the buffer back to the pointer yields the same value, and the pointer is again valid and points to the same object.

Example 6: destroy and recreate the object

[Compiler explorer link](#)

```
#include <assert.h>
#include <string.h>
#include <stdio.h>
#include <new>

struct X {
    int i;
};

int main() {
    X *x= new X{42};
    X *y= nullptr;
    unsigned char buffer[sizeof(x)];
    memcpy(buffer, &x, sizeof(x));

    x->~X();
    new(x) X{99};

    memcpy(&y, buffer, sizeof(x));
    assert(x == y);
    assert(y->i == 99);
    assert(x->i == 99);
}
```

```
}
```

This time, we destroy the pointed-to object and recreate a new object with a new value at the same memory location.

The pointer `x` still holds the same bit pattern, and still points to a valid object, so both the original pointer `x` and the newly constructed copy `y` point to the new object, and all is well by [basic.life] p8.

Example 7: delete and new the object

[Compiler explorer link](#)

```
#include <assert.h>
#include <string.h>
#include <stdio.h>
#include <new>

struct X {
    int i;
};

int main() {
    X *x= new X{42};
    X *y= nullptr;
    unsigned char buffer[sizeof(x)];
    memcpy(buffer, &x, sizeof(x));

    delete x;
    y= new X{99};

    unsigned char buffer2[sizeof(x)];
    memcpy(buffer2, &y, sizeof(x));

    if(memcmp(buffer, buffer2, sizeof(x))) {
        printf("Different address\n");
        return 0;
    }

    memcpy(&x, buffer2, sizeof(x));

    assert(x == y);
    assert(y->i == 99);
    assert(x->i == 99);
}
```

This time, we destroy the pointed-to object with `delete` and recreate a new object with a new value with `new`.

We then copy the new pointer into a buffer and compare the buffers. If the buffers are different, then the pointers are clearly different and our test doesn't work, so we stop.

If the buffers are the same, then we copy the new buffer (which is a copy of our new pointer) into the old pointer.

`x` is now a copy of the raw bits of our new pointer, so everything must work.

Example 8: delete and new the object again

[Compiler explorer link](#)

```

#include <assert.h>
#include <string.h>
#include <stdio.h>
#include <new>

struct X {
    int i;
};

int main() {
    X *x= new X{42};
    X *y= nullptr;
    unsigned char buffer[sizeof(x)];
    memcpy(buffer, &x, sizeof(x));

    delete x;
    y= new X{99};

    unsigned char buffer2[sizeof(x)];
    memcpy(buffer2, &y, sizeof(x));

    if(memcmp(buffer, buffer2, sizeof(x))) {
        printf("Different address\n");
        return 0;
    }

    assert(x == y);
    assert(y->i == 99);
    assert(x->i == 99);
}

```

This is the same as example 7, except we don't copy the raw bits from the new buffer over our old pointer.

We know that the bits of `x` and the bits of `y` are the same because we compared them with `memcmp`. Since the pointers are trivially copyable, the value of the pointer is determined by the *value representation*, which is the set of bits of the *object representation*. Since we know the *object representation* is the same, the *value representation* must be the same, so the pointers must have the same value.

Since the pointers must have the same value, `x` must be equal to `y`, and must point to the same object, and all is well.

Example 9: using `std::atomic` to hold the pointer

[Compiler explorer link](#)

```

#include <assert.h>
#include <string.h>
#include <stdio.h>
#include <new>
#include <atomic>

struct X {
    int i;
};

int main() {
    X *x= new X{42};
    X *y= nullptr;
    std::atomic<X *> p(x);

    delete x;
}

```

```

y= new X{99};

X *temp= y;
if(!p.compare_exchange_strong(temp, y)) {
    printf("Different address\n");
    return 0;
}

assert(x == y);
assert(y->i == 99);
assert(x->i == 99);
}

```

This is the same as example 8, except instead of using `memcmp` to determine the equivalence, we use `compare_exchange_strong`, which compares pointer as-if with `memcmp`.

Example 10: using `std::atomic` to hold the pointer, comparison the other way round

[Compiler explorer link](#)

```

#include <assert.h>
#include <string.h>
#include <stdio.h>
#include <new>
#include <atomic>

struct X {
    int i;
};

int main() {
    X *x= new X{42};
    X *y= nullptr;

    delete x;
    y= new X{99};

    std::atomic<X *> p(y);
    if(!p.compare_exchange_strong(x, nullptr)) {
        printf("Different address\n");
        return 0;
    }

    assert(x == y);
    assert(y->i == 99);
    assert(x->i == 99);
}

```

This is the same as example 9, except that rather than comparing the temp value copied from y with our stored pointer, we store the new value in the atomic, and compare it to our original x. This still works because the `compare_exchange_strong` compares as-if using `memcmp`, so we are comparing the object representation of x against the object representation of the copy of y stored in p: if the pointers have the same object representation then they have the same value representation, so must be the same and point to the same object.

Acknowledgements

All standard references are to the C++ working draft from the 2020-04 mailing: [N4861](#).

Thanks to Richard Smith, Hubert Tong, Peter Sewell, Martin Eucker, Peter Dimov, Jens Gustedt, Hans Boehm, Jens Maurer, Roger Orr, Ville Voutilainen, Bronek Kozicki, Balog Pal, Andrey Erokhin, Niall Douglas, Gabriel dos Reis, Olivier Giroux, Caleb Substrum, Alsdair Meredith, Nathan Myers, Bjarne Stroustrup, Nevin Liber, JF Bastien, Paul McKenney, Maged Michael, and others for their comments on the first revision of this paper.